

The Request-Response Cycle (CodeGrade)

 [. \(https://github.com/learn-co-curriculum/python-p4-request-response-cycle\)](https://github.com/learn-co-curriculum/python-p4-request-response-cycle)  [. \(https://github.com/learn-co-curriculum/python-p4-request-response-cycle/issues/new\)](https://github.com/learn-co-curriculum/python-p4-request-response-cycle/issues/new)

Learning Goals

- Build and run a Flask application on your computer.
 - Manipulate and test the structure of a request object.
-

Key Vocab

- **Web Framework:** software that is designed to support the development of web applications. Web frameworks provide built-in tools for generating web servers, turning Python objects into HTML, and more.
 - **Extension:** a package or module that adds functionality to a Flask application that it does not have by default.
 - **Request:** an attempt by one machine to contact another over the internet.
 - **Client:** an application or machine that accesses services being provided by a server through the internet.
 - **Web Server:** a combination of software and hardware that uses Hypertext Transfer Protocol (HTTP) and other protocols to respond to requests made over the internet.
 - **Web Server Gateway Interface (WSGI):** an interface between web servers and applications.
 - **Template Engine:** software that takes in strings with tokenized values, replacing the tokens with their values as output in a web browser.
-

Introduction

Web browsers cannot execute Python code. To make our views show up in the browser, Flask needs to translate HTTP requests into Python objects and new Python objects into HTTP responses. There are plenty of strategies to do this by hand, but Flask makes our lives easier with Werkzeug- you'll find that Flask generates WSGI maps for your application with little to no manual configuration on your part.

Let's take a look under the hood at the request-response cycle in Flask applications.

Application and Request Contexts

When a Flask application gets a request from the browser, it has to pass some specific objects to the view function that will respond to that request. One example is the `request` object itself, which contains the HTTP request from the browser.

You may have noticed in the past few lessons, though, that view functions do not take `request` objects as arguments. Since every view function needs access to a `request` object- among other things- Flask manages requests through **contexts**.

Let's take a look at a simple view function that uses a request object:

```
# server/app.py

from flask import Flask, request

app = Flask(__name__)

@app.route('/')
def index():
    host = request.headers.get('Host')
    return f'<h1>The host for this page is {host}</h1>'

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Remember to set the environment variables in the `server/` directory as well:

```
$ export FLASK_APP=app.py  
$ export FLASK_RUN_PORT=5555
```

Running this app and navigating to `127.0.0.1:5555`, you should see the following:

A screenshot of a web browser's address bar. It features navigation icons (back, forward, refresh) and a text input field containing 'localhost:5555'.

The host for this page is localhost:5555

This might seem a bit odd. While we did import `request`, we didn't assign it any attributes that would tell it about the activity on our server. It almost seems like a **global** variable- one that is set outside of our application instances and views that provides an unchanging set of information. That wouldn't make much sense as an implementation, though: our application will be handling many requests, often overlapping with one another while multiple users are visiting our website.

Flask generates a context for requests after receiving a request and before our application runs. When `request` is called, we have access to all of that request data without having to do any configuration or include a `request` argument to our view.

If you're testing `request` for a specific app outside of any views, you will have to generate this context manually. This only requires two commands:

```
# example only- no need to write this out
```

```
from app import app  
from flask import request
```

```
request_context = app.test_request_context()  
request_context.push()
```

NOTE: If you are working in a debugging shell, it is a good idea to clear out your `request_context` object with its `pop()` method before moving onto a new request.

As the name `test_request_context()` suggests, this only provides test data. You won't use this in your applications, but you might see it in the tests for Phase 4's labs.

Flask also has an **application context**. This works very much the same way as a request context: when a request is received, Flask generates the application context. When the application instance is accessed, the application context becomes available to our app and views. This provides information on the application we're working on and is accessible through the `flask.current_app` object.

```
# server/app.py

from flask import Flask, request, current_app

app = Flask(__name__)

@app.route('/')
def index():
    host = request.headers.get('Host')
    appname = current_app.name
    return f'''<h1>The host for this page is {host}</h1>
               <h2>The name of this application is {appname}</h2>'''

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Flask also provides us two unique objects that allow us to manipulate request data more effectively:

- `g` is an object that can be used to store anything that you want to store globally for the lifetime of a request. It is reset with each new request.
- `session` is a dictionary object that can be used to hold onto values between multiple requests.

Handling Requests

Now that we've seen how a request object is created, let's take a look at how requests are used by Flask applications.

The URL Map

Every time an application receives a request, it has to decide which view to run to handle its data. Routes were helpful for defining where to serve our application's views, but now we need to use our application's *URL map* to find the right view from a URL. The URL map is just as it sounds: a dictionary that maps URLs to the views that serve the client at those URLs. Every time we use the `@app.route` decorator, a new mapping is added to the URL map.

We can view the URL map for ourselves in the Flask shell. Run `flask shell` from the `server/` directory for this lesson and enter the following:

```
$ app.url_map
# => Map([<Rule '/static/<filename>' (GET, HEAD, OPTIONS) -> static>,
  <Rule '/' (GET, HEAD, OPTIONS) -> index>])
```

The first `Rule` we see exists by default: it is a special route that gives us access to static files through the client. The second, referring to the `'/'` URL, describes the `index()` view that we made above.

`GET`, `HEAD`, and `OPTIONS` are request methods that are accepted through the routes. All HTTP (*and HTTPS*) requests must be issued with a request method to indicate what task the server is meant to carry out. `GET` requests a representation of the resource- this is the most common method your requests carry when you're surfing the web. `HEAD` is the same as `GET`, but requests that we leave out the body of the response. `OPTIONS` requests a list of the HTTP methods that a resource will accept. Flask will include all three by default- but you can remove them or add more later on if you'd like!

For more on HTTP request methods, visit [Mozilla HTTP Request Methods][moz_http].

Request Hooks

As you build out a variety of Flask web applications, you will notice that there are many tasks that you want to carry out before or after most of your view functions. This could be as simple as generating a reminder message or as complex as multi-factor authentication- either way, you'll want to handle these with **hooks**.

Hooks are best implemented as decorators. There are four types of hooks:

1. `@app.before_request` : runs a function before each request.
2. `@app.before_first_request` : runs a function before the first request (but not subsequent requests).
3. `@app.after_request` : runs a function after each request.
4. `@app.teardown_request` : runs a function after each request, even if an error has occurred.

Let's set up a hook so that our views all know where our application files are located:

```
# server/app.py

import os

from flask import Flask, request, current_app, g

app = Flask(__name__)

@app.before_request
def app_path():
    g.path = os.path.abspath(os.getcwd())

@app.route('/')
def index():
    host = request.headers.get('Host')
    appname = current_app.name
    return f'''<h1>The host for this page is {host}</h1>
    <h2>The name of this application is {appname}</h2>
    <h3>The path of this application on the user's device is {g.path}</h3>'''
```

```
if __name__ == '__main__':  
    app.run(port=5555, debug=True)
```

After you restart your server, you should see that our hook has been run and `g` modified such that `index()` now knows where it lives on your computer. *(Now it can find its way home if it ever gets lost!)*

Creating Responses

When a view function spins up, Flask gets ready for an HTTP response as a return value. This can be a simple string, a multi-line string of HTML, or a combination of strings and codes.

An important part of any response is the HTTP **status code**. Flask sets this to 200 by default, which indicates that the request successfully reached the specified resource and an appropriate response was generated. When we need to send a different status code, we can simply add this as a second return value after the response body:

```
# index() in server/app.py
```

```
@app.route('/')  
def index():  
    host = request.headers.get('Host')  
    appname = current_app.name  
    return f'''<h1>The host for this page is {host}</h1>  
           <h2>The name of this application is {appname}</h2>  
           <h3>The path of this application on the user's device is {g.path}</h3>''', \  
           202
```

202 is the "Accepted" status code. This signifies that a request has been received by the server, but that the server has not done anything about it yet. We could also return 204 if there were no content on the page, or 404 if the URL was not found.

For more on HTTP status codes, visit the [Mozilla documentation](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status)  (<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>).

There is a third, optional argument that can be added in to create headers for our response. This is simply a dictionary with keys for the header attributes mapped to their respective values.

Response Objects

For a more object-oriented approach to responses, you can use Flask's `make_response()` function. This takes 1-3 arguments in the same format as our earlier response: a body string, a status code, and a headers dictionary, respectively.

```
# index() in server/app.py
...
from flask import make_response
...

@app.route('/')
def index():
    host = request.headers.get('Host')
    appname = current_app.name
    response_body = f'''
        <h1>The host for this page is {host}</h1>
        <h2>The name of this application is {appname}</h2>
        <h3>The path of this application on the user's device is {g.path}</h3>
    '''

    status_code = 200
    headers = {}

    return make_response(response_body, status_code, headers)
```

This won't change what you see in the browser, but it will make your code cleaner and easier to replicate (*even automate!*) in other views.

For more on response objects, visit [API Flask Documentation](https://flask.palletsprojects.com/en/2.2.x/api/#response-objects)  <https://flask.palletsprojects.com/en/2.2.x/api/#response-objects>.

Special Responses

There are specific cases where your response is meant to do something other than display an HTML body in the browser. Two cases are most common: redirects and aborts.

The `redirect()` function is usually delivered with a "301: Moved Permanently" or "302: Found" status code. These signify that the URL for the resource on our server has been changed. Rather than torture the user with trying to find the new URL, we can redirect them to the current URL for the resource.

`redirect()` is very simple: it takes one argument, the URL for the relocated resource.

example only

```
from flask import redirect

@app.route('/reginald-kenneth-dwight')
def index():
    return redirect('names.com/elton-john')
```

If you navigate to the above link, you'll notice a "404: Not Found" status code! This means that the resource does not exist for the website in question. If we want to inform users of this error in our applications, we need to use Flask's `abort()` function:

example only

```
...
from flask import abort, make_response
...

@app.route('/<stage_name>')
def get_name(stage_name):
```

```
match = session.query('StageName').filter(StageName.name == stage_name)[0]
if not match:
    abort(404)
return make_response(f'<h1>{stage_name} is an existing stage name!</h1>')
```

Conclusion

This has been a brief introduction to requests and responses in Flask. These messages allow our application to communicate with a client through the server. Flask does most of the work for us when setting up contexts and URL maps, but parsing requests and generating responses effectively will allow you to do much more with Flask than you've seen so far. Now that we've discussed communication with the frontend, let's get started on communication with the backend through SQLAlchemy.

If you don't fully understand every concept yet, don't worry! Check below to make sure your code matches ours and look back at this lesson if you're having trouble with contexts, requests, or responses in the future.

Solution Code

```
#!/usr/bin/env python3

import os

from flask import Flask, request, current_app, g, make_response

app = Flask(__name__)

@app.before_request
def app_path():
    g.path = os.path.abspath(os.getcwd())
```

```
@app.route('/')
def index():
    host = request.headers.get('Host')
    appname = current_app.name
    response_body = f'''
        <h1>The host for this page is {host}</h1>
        <h2>The name of this application is {appname}</h2>
        <h3>The path of this application on the user's device is {g.path}</h3>
    '''

    status_code = 200
    headers = {}

    return make_response(response_body, status_code, headers)

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- [API - Pallets Projects](https://flask.palletsprojects.com/en/2.2.x/api/) ➞ (https://flask.palletsprojects.com/en/2.2.x/api/)
- [HTTP request methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)
- [HTTP response status codes - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Status)
- [Response Objects - Pallets Projects](https://flask.palletsprojects.com/en/2.2.x/api/#response-objects) ➞ (https://flask.palletsprojects.com/en/2.2.x/api/#response-objects)

This tool needs to be loaded in a new browser window

Load The Request-Response Cycle (CodeGrade) in a new window

